SPECIFICATION

Background of the Invention

Conventional secure communication protocols, from Transport Layer Security (TLS) to public-key cryptography, operate on a sequential, "key-first" paradigm. A secure channel is established, cryptographic keys are exchanged (e.g., via Diffie-Hellman or RSA), and only then is the encrypted data transmitted. This model's primary vulnerability is the key exchange itself. An attacker who can intercept the key exchange in real-time (a "Man-in-the-Middle" or MITM attack) can often compromise the entire session that follows. The success of such an attack is predicated on the temporal and spatial collision of the attacker with a single, critical event: the key exchange. There is a need for a communication method that is not vulnerable to a single-point-of-failure interception.

Summary of the Invention

The present invention provides a method for secure communication that decouples the transmission of the data from the transmission of the final cryptographic key, providing "backwards secrecy." Instead of sending the key first, the data is sent in multiple parts, and the key required for decryption is the last element to be transmitted.

The process begins with a sender generating a first, secret key (K1) which is never transmitted. The core message (M) is encrypted with K1 and transmitted to a receiver. At a subsequent time, and optionally over a different communication channel, the sender transmits a prefix (P). At another subsequent time, the sender transmits a suffix (S). With the three data components now transmitted, the sender computes a second, final key (K2) based

on K1 and the fully assembled data block (P+M+S). Finally, at another subsequent time, the sender transmits K2 to the receiver. The receiver assembles the data block and uses the received K2 to decrypt it. This method drastically increases the difficulty for an attacker, who must now intercept four distinct transmissions and correctly identify their relationship without any contextual clues, and crucially, the transmitted key (K2) is only viable to decrypt the fully assembled data block (P+M+S) but not P, M, or S individually, or in other words, the message transmitted (M encrypted using key K1) cannot be decrypted with the key transmitted (K2).

Brief Description of the Drawings

FIG. 1 is a flowchart illustrating the sequence of transmissions and key derivation steps of the secure communication method, showing the asynchronous interactions between the sender and receiver.

Detailed Description of the Invention

Referring to **FIG. 1**, the method for asynchronous secure communication is illustrated. The process is divided between a Sender (101) and a Receiver (102).

The process begins at the Sender (101). In step (110), an initial, internal-only secret key, K1, is generated. This key is never transmitted in any form. In step (112), the core message, M, is encrypted using a symmetric encryption algorithm with K1 as the key.

In step (114), the encrypted message M is transmitted to the Receiver (102), who receives and stores it (150). Subsequently, in steps (116) and (118), the Sender transmits a prefix P and a suffix S, respectively. These transmissions are asynchronous, meaning they can

occur at different times and optionally over different communication channels (e.g., M over satellite, P over fiber, S via a dead drop). The Receiver (102) collects and stores P and S as they arrive (152, 154).

After the three data components have been sent, the Sender (101) proceeds to step (120). The Sender concatenates the transmitted components in the correct order to form a final message block (P+M+S). A second key, K2, is then computed using a deterministic process, for example, by using a Key Derivation Function (KDF) that takes K1 and a cryptographic hash of the final message block as input (e.g., K2 = KDF(K1, SHA-256(P+M+S))).

Finally, in step (122), the Sender transmits the final key, K2. The Receiver (102) receives K2 (156). With all four components now received, the Receiver proceeds to the final step (158). The Receiver assembles the message block (P+M+S) in the correct order and uses the received key K2 to decrypt it, recovering the original plaintext message.